Marevlo Research / AI Foundations • Module 06 – Pruning, Distillation & MoE • Topic 3

Knowledge Distillation, Worked Out by Hand

Soft targets, temperature, "dark knowledge," and the mysterious T^2

What this document does. It takes one position — a teacher and a student each emitting five logits over a tiny vocabulary — and walks the entire distillation loss by hand. We soften both distributions at temperature $T = 2$, compute the KL term that pulls the student toward the teacher, assemble the full composite loss $L = \alpha\,\mathrm{CE} + \beta\,T^2\,\mathrm{KL} + \gamma\,\mathrm{MSE}$, and finally derive *why* the T^2 factor has to be there. Every number is produced by a short NumPy script (Appendix A, `m06_t03_distillation_engine.py`).

The one idea. A trained teacher's full output distribution carries far more information than its single argmax token. The relative probabilities it assigns to the *runner-up* tokens — "this was almost a 5, definitely not a 2" — are the "dark knowledge." Distillation transfers that whole distribution, not just the label, and temperature is the dial that exposes it.

Concepts covered, in order
teacher/student logits; the hard (T=1) softmax and its peakedness; temperature softening; dark knowledge; the KL distillation term; the composite loss with weights α, β, γ; the feature-matching MSE term; and the $1/T^2$ gradient-scale argument that motivates the T^2 multiplier.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

Contents

1 The setup — one position, five tokens

A real distillation runs this at every position over a 128k-token vocabulary across billions of tokens. We fix one position and a five-token vocabulary so the softmaxes are checkable by hand.

Quantity	Symbol	Value here
Teacher logits	z_t	(4.0, 2.0, 1.0, 0.5, −1.0)
Student logits	z_s	(3.0, 1.0, 1.5, 0.0, −0.5)
Ground-truth token	y	class 0
Temperature	T	2.0
Loss weights	α, β, γ	0.3, 0.6, 0.1

Intuition

The teacher and student largely agree the answer is token 0. But look at tokens 2 and 3: the teacher ranks token 2 (z=1.0) above token 3 (z=0.5), while the student has *flipped* them (z_s: token 2=1.5, token 3=0.0). Hard-label training (just "the answer is 0") would never tell the student about this disagreement on the losers. Distillation does.

2 Step 1 — The hard view ($T = 1$)

Standard softmax, $p_i = e^{z_i} / \sum_j e^{z_j}$:

$p_t(T{=}1) = (0.818, 0.111, 0.041, 0.025, 0.005), \quad p_s(T{=}1) = (0.695, 0.094, 0.155, 0.035, 0.021).$

The teacher is sharply peaked: 81.8% of its mass sits on the top token. The remaining 18.2% — spread over tokens 1–4 — is the dark knowledge. But at $T = 1$ that signal is faint: tokens 3 and 4 get 2.5% and 0.5%, almost invisible to a gradient.

Watch out

If you distill at $T = 1$, the loss is dominated by the one big probability and the informative tail is numerically negligible. The student learns "token 0" — which it could have learned from the hard label anyway. You have thrown away the very thing distillation is for.

3 Step 2 — Temperature softening ($T = 2$)

Divide every logit by T *before* the softmax:

Temperature-softened softmax

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

Higher T shrinks the logit gaps, flattening the distribution toward uniform.

At $T = 2$:

$p_t(T{=}2) = (0.542, 0.199, 0.121, 0.094, 0.044), \quad p_s(T{=}2) = (0.447, 0.164, 0.211, 0.100, 0.078).$

The teacher's top-token mass falls from 81.8% to 54.2%. The runner-ups swell: token 1 goes 11.1% → 19.9%, token 3 goes 2.5% → 9.4%. The dark knowledge is now loud enough to carry a gradient.

What temperature actually does

T is a magnifying glass on the loser tokens. At $T \to 0$ the softmax becomes a hard argmax (all mass on one token); at $T \to \infty$ it becomes uniform (every token equal). Somewhere in between — $T = 2$ to 4 in practice — the *relative ranking and spacing* of the non-top tokens is most visible, and that ranking is the richest part of the teacher's signal.

4 Step 3 — The KL distillation term

We pull the student's softened distribution toward the teacher's with a Kullback–Leibler divergence:

Soft-target matching loss

$$\mathrm{KL}\big(p_t^{(T)} \,\|\, p_s^{(T)}\big) = \sum_i p_{t,i}^{(T)} \, \log \frac{p_{t,i}^{(T)}}{p_{s,i}^{(T)}}.$$

Plugging in the $T = 2$ vectors term by term and summing:

$$\mathrm{KL}\big(p_t^{(2)} \,\|\, p_s^{(2)}\big) = 0.04424 \text{ nats.} \quad \checkmark$$

This is the per-position penalty for the student disagreeing with the teacher's *whole* softened distribution — heavily weighted toward where the teacher is confident, but still accounting for the tail. (For contrast, at $T = 1$ the same KL is 0.0813; the softening both reveals the tail *and* changes the loss surface.)

5 Step 4 — The composite loss

Production distillation blends three signals:

Composite distillation loss

$$L = \underbrace{\alpha \, \mathrm{CE}(y,\, p_s)}_{\text{hard label}} + \underbrace{\beta \, T^2 \, \mathrm{KL}\big(p_t^{(T)} \,\|\, p_s^{(T)}\big)}_{\text{soft targets}} + \underbrace{\gamma \textstyle\sum_l \mathrm{MSE}\big(h_s^l,\, W_l h_t^l\big)}_{\text{feature match}}.$$

Typical: α=0.3, β=0.6, γ=0.1, T=2.

Assemble the three terms

Hard-label CE (student at T=1, true class 0 with $p_{s,0} = 0.695$):

$$\mathrm{CE} = -\log(0.695) = 0.3636, \qquad \alpha \, \mathrm{CE} = 0.3 \times 0.3636 = 0.1091.$$

Soft KL (from Step 3, with the $T^2 = 4$ multiplier):

$$\beta \, T^2 \, \mathrm{KL} = 0.6 \times 4 \times 0.04424 = 0.1062.$$

Feature MSE (one hidden vector, student vs. projected teacher hidden; MSE = 0.00437):

$$\gamma \, \mathrm{MSE} = 0.1 \times 0.00437 = 0.00044.$$

Total:

$$L = 0.1091 + 0.1062 + 0.00044 = 0.2157. \quad \checkmark$$

Notice the T^2 multiplier just made the soft-target term (0.106) comparable in size to the hard-label term (0.109) — exactly the balance α and β were chosen for. Without it the soft term would be a quarter as large and the student would barely listen to the teacher. That is the cue for the next step.

Intuition

The three terms answer three questions. CE: "did you get the answer right?" KL: "did you get the teacher's whole opinion right, including the losers?" MSE: "do your internal representations look like the teacher's?" The first needs only a label; the second and third need a white-box teacher and are where the real compression magic lives.

6 Step 5 — *Why* the T^2 factor exists

The T^2 is not cosmetic; it cancels a $1/T^2$ shrinkage in the gradients. The gradient of the KL term with respect to a student logit is

Soft-target gradient

$$\frac{\partial}{\partial z_{s,i}} \mathrm{KL}\big(p_t^{(T)} \,\|\, p_s^{(T)}\big) = \frac{1}{T}\big(p_{s,i}^{(T)} - p_{t,i}^{(T)}\big).$$

There are *two* factors of T hiding here: an explicit $1/T$ out front, and a second $1/T$ because softening at large T makes the probability *difference* $(p_s - p_t)$ itself shrink (both flatten toward uniform). Net effect: the raw soft-target gradient scales like $1/T^2$.

Watch the gradient shrink, then get restored

Gradient L_2-norm $\|\partial \mathrm{KL}/\partial z_s\|$ at three temperatures, and the same after multiplying by T^2:

	$T=1$	$T=2$	$T=4$
raw $\|\nabla\|$	0.1699	0.0697	0.0189
(ratio to T=1)	1.00	0.41	0.11
$\times T^2$	0.1699	0.2788	0.3024

The raw norm collapses by $\sim 1/T^2$ ($0.41 \approx 1/2^2$, $0.11 \approx 1/4^2$ ballpark). Multiplying the loss by T^2 lifts it back to the same order of magnitude as the hard-label gradient, so changing T tunes *what* the student sees without silently turning the soft loss off. ✓

The T^2 in one sentence

Softening by T shrinks soft-target gradients by $1/T^2$; the explicit T^2 in the loss puts them back, so the temperature controls the *information* in the soft targets without also controlling their *learning rate*. The two knobs stay independent.

7 The argument, at a glance

#	Step	Result
1	Hard view	teacher 81.8% on top token; tail faint
2	Soften	T=2: top mass $\to$ 54.2%, tail revealed
3	KL term	$\mathrm{KL}(p_t \| p_s) = 0.0442$ nats
4	Composite	$0.109 + 0.106 + 0.0004 = 0.2157$
5	T^2 why	raw grad $\propto 1/T^2$; T^2 restores it ✓

7.1 Equation cheat-sheet

Softened softmax	$p_i^{(T)} = \exp(z_i/T) / \sum_j \exp(z_j/T)$
KL soft target	$\mathrm{KL}(p_t^{(T)} \| p_s^{(T)}) = \sum_i p_{t,i}^{(T)} \log \frac{p_{t,i}^{(T)}}{p_{s,i}^{(T)}}$
Hard label	$\mathrm{CE}(y, p_s) = -\log p_{s,y}$
Composite loss	$L = \alpha\,\mathrm{CE} + \beta\,T^2\,\mathrm{KL} + \gamma\,\mathrm{MSE}$
Soft-target grad	$\partial_{z_s}\mathrm{KL} = \frac{1}{T}(p_s^{(T)} - p_t^{(T)})$
Grad scale	$\|\nabla_{\text{raw}}\| \propto 1/T^2 \;\Rightarrow$ multiply by T^2

Appendix A — Reference implementation

Every number above is produced by `m06_t03_distillation_engine.py`. Run it to reproduce the softmaxes, the KL term, the composite loss, and the gradient-scale table; change T or the logits to explore.

```
"""
Reference implementation for: Module 06 - Topic 3
"Knowledge distillation - soft targets, temperature, and the T^2 factor."

Worked on a tiny 5-class logit vector (think: 5-token vocabulary, one position).
We compute:
  - teacher & student softmax at T=1 (hard view) and T=2 (softened view)
  - the KL distillation term KL(p_teacher^T || p_student^T)
  - the full composite loss L = a*CE + b*T^2*KL + c*MSE(hidden)
  - the gradient-scale argument for why the beta*T^2 factor exists.
"""

import numpy as np
np.set_printoptions(precision=4, suppress=True)

def softmax(z, T=1.0):
    z = np.asarray(z, dtype=float) / T
    z = z - z.max()
    e = np.exp(z)
    return e / e.sum()

def kl(p, q):
    # KL(p || q) = sum p * log(p/q)
    p = np.clip(p, 1e-12, 1.0)
    q = np.clip(q, 1e-12, 1.0)
    return float(np.sum(p * np.log(p / q)))

def cross_entropy(y_onehot, p):
    p = np.clip(p, 1e-12, 1.0)
```

```
    return float(-np.sum(y_onehot * np.log(p)))

# ------------------------------------------------------------------------
# 0. One position: teacher and student logits over a 5-token vocabulary.
# ------------------------------------------------------------------------
z_t = np.array([4.0, 2.0, 1.0, 0.5, -1.0])   # teacher logits
z_s = np.array([3.0, 1.0, 1.5, 0.0, -0.5])   # student logits
y_true = 0                                    # ground-truth token index
y_onehot = np.eye(len(z_t))[y_true]
T = 2.0
print("== 0. Logits ==")
print("teacher z_t =", z_t)
print("student z_s =", z_s)
print("ground-truth class =", y_true)

# ------------------------------------------------------------------------
# 1. Hard view (T=1): the argmax + how peaked the teacher is.
# ------------------------------------------------------------------------
pt1 = softmax(z_t, T=1.0)
ps1 = softmax(z_s, T=1.0)
print("\n== 1. T=1 (hard softmax) ==")
print("p_teacher(T=1) =", pt1)
print("p_student(T=1) =", ps1)
print(f"teacher puts {pt1[0]*100:.1f}% on the top token; rest is 'dark knowledge'")

# ------------------------------------------------------------------------
# 2. Soft view (T=2): temperature flattens the distribution.
# ------------------------------------------------------------------------
pt2 = softmax(z_t, T=T)
ps2 = softmax(z_s, T=T)
print("\n== 2. T=2 (softened softmax) ==")
print("p_teacher(T=2) =", pt2)
print("p_student(T=2) =", ps2)
print(f"top token mass {pt1[0]*100:.1f}% (T=1) -> {pt2[0]*100:.1f}% (T=2): "
      "low-prob tokens revealed")

# ------------------------------------------------------------------------
# 3. The KL distillation term at T=2.
# ------------------------------------------------------------------------
kl_T2 = kl(pt2, ps2)
kl_T1 = kl(pt1, ps1)
print("\n== 3. KL( p_teacher || p_student ) ==")
print(f"KL at T=1 = {kl_T1:.5f} nats")
print(f"KL at T=2 = {kl_T2:.5f} nats")

# ------------------------------------------------------------------------
# 4. The composite loss.
#    L = alpha*CE(y,p_student@T=1) + beta*T^2*KL(pt@T || ps@T) + gamma*MSE(hidden)
# ------------------------------------------------------------------------
alpha, beta, gamma = 0.3, 0.6, 0.1
ce = cross_entropy(y_onehot, ps1)
# tiny hidden-state feature-match example (student vs projected teacher hidden)
h_student = np.array([0.20, -0.10, 0.50, 0.30])
h_teacher_proj = np.array([0.25, -0.05, 0.40, 0.35])
mse = float(np.mean((h_student - h_teacher_proj) ** 2))
L_ce   = alpha * ce
L_kl   = beta * (T**2) * kl_T2
L_mse  = gamma * mse
L_total = L_ce + L_kl + L_mse
print("\n== 4. Composite distillation loss (alpha=0.3, beta=0.6, gamma=0.1, T=2) ==")
print(f"CE(y_true, p_student@T=1)      = {ce:.5f}   -> alpha*CE   = {L_ce:.5f}")
print(f"KL term @T=2                    = {kl_T2:.5f}   -> beta*T^2*KL= {L_kl:.5f}")
print(f"MSE(hidden)                    = {mse:.5f}   -> gamma*MSE  = {L_mse:.5f}")
```

```
print(f"L_total                        = {L_total:.5f}")

# ---------------------------------------------------------------------
# 5. Why the T^2 factor: soft-target gradients scale as 1/T^2.
#    Restore them with an explicit T^2 so the KL term keeps a steady
#    gradient magnitude as T changes. Demonstrate numerically.
# ---------------------------------------------------------------------
def kl_grad_wrt_student_logits(z_s, z_t, T):
    # d/dz_s KL(softmax(z_t/T) || softmax(z_s/T)) = (1/T)*(p_student^T - p_teacher^T)
    return (softmax(z_s, T) - softmax(z_t, T)) / T

print("\n== 5. The 1/T^2 gradient-scale argument ==")
for Tt in [1.0, 2.0, 4.0]:
    g = kl_grad_wrt_student_logits(z_s, z_t, Tt)
    gnorm = np.linalg.norm(g)
    print(f"T={Tt}:  ||dKL/dz_s|| = {gnorm:.5f}   "
          f"after *T^2: {gnorm * Tt**2:.5f}")
print("Raw soft-target gradient shrinks ~1/T^2; multiplying the KL term by T^2")
print("rescales it back so soft and hard losses stay balanced as T varies.")
```

— END OF DOCUMENT —